

The background features a dark blue globe with a glowing yellow network overlay. The network consists of numerous interconnected nodes and lines, forming a complex web that covers the globe. The nodes are small yellow dots, and the lines are thin yellow lines. The globe itself is a darker blue with some lighter blue highlights. The overall aesthetic is high-tech and digital.

一小时深入浅出高并发

开放平台技术沙龙第三期

目录

Content

- 1 如何定义高并发？
- 2 高并发为什么难以解决？
- 3 双十一背后发生了什么？
- 4 那些年，我们遇到过的高并发
- 5 有没有一套完整的解决方案？

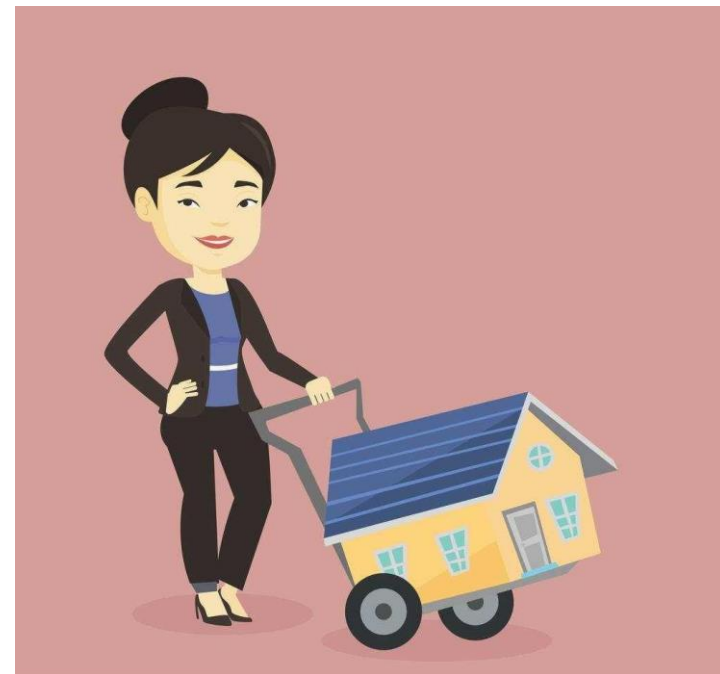
1

如何定义高并发？

高并发定义

楼盘开盘，茫茫人海，**你**去纸上写自己心仪的房号，

你能不能挤进去？即使挤进去一定就能买到**自己相中**
的房间吗？



挤

同一时刻产生茫茫多的意图（流量、请求、进程、线程等）



抢

对于有限资源（带宽、CPU、内存、磁盘等）无节制的索取



什么时候会出现高并发？



营销活动
(双十一、红包雨)



秒杀



不合理实现导致的频繁调用



恶意攻击

典型问题 -- 拒绝服务

服务器端的配置

阿里云服务器(ecs.xn4.small)

CentOS 7、1核1G

1Mbps

mariadb

Apache

PHP

使用Jmeter向测试IP大量发送请求



典型问题 -- 拒绝服务

Jmeter的汇总报告

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received ...	Sent KB/sec	Avg. Bytes
HTTP请求	2058	15213	48	16560	2131.91	97.18%	123.0/sec	379.99	0.40	3162.2
总体	2058	15213	48	16560	2131.91	97.18%	123.0/sec	379.99	0.40	3162.2

返回503 error

服务器的httpd的error_log，服务器的mariadb挂掉了

```
[ :error] [pid 19405] [client 117.71.53.46:53535] Lost connection to MySQL server during query
[ :error] [pid 19344] [client 117.71.53.46:53604] Lost connection to MySQL server during query
[ :error] [pid 19357] [client 117.71.53.46:53728] Lost connection to MySQL server during query
[ :error] [pid 19338] [client 117.71.53.46:53589] Lost connection to MySQL server during query
[ :error] [pid 19310] [client 117.71.53.46:53520] Lost connection to MySQL server during query
[ :error] [pid 19431] [client 117.71.53.46:53547] Lost connection to MySQL server during query
```

典型问题 -- 资源竞争死锁

```
@Slf4j
public class DeadLock implements Runnable {
    public int flag = 1;
    //静态对象是类的所有对象共享的
    private static Object o1 = new Object(), o2 = new Object();

    @Override
    public void run() {
        log.info("flag:{}", flag);
        if (flag == 1) {
            synchronized (o1) {
                try {
                    Thread.sleep( millis: 500);
                } catch (Exception e) {
                    e.printStackTrace();
                }
                synchronized (o2) {
                    log.info("1");
                }
            }
        }
        if (flag == 0) {
            synchronized (o2) {
                try {
                    Thread.sleep( millis: 500);
                } catch (Exception e) {
                    e.printStackTrace();
                }
                synchronized (o1) {
                    log.info("0");
                }
            }
        }
    }
}
```

一个简单的死锁Demo

```
public static void main(String[] args) {
    DeadLock td1 = new DeadLock();
    DeadLock td2 = new DeadLock();
    td1.flag = 1;
    td2.flag = 0;
    //td1,td2都处于可执行状态，但JVM线程调度先执行哪个线程是不确定的。
    //td2的run()可能在td1的run()之前运行
    new Thread(td1).start();
    new Thread(td2).start();
}
```

典型问题 -- 数据不一致

```
@Slf4j
public class CountExample {

    // 请求总数
    public static int clientTotal = 5000;

    // 同时并发执行的线程数
    public static int threadTotal = 200;

    public static int count = 0;

    public static void main(String[] args) throws Exception {
        ExecutorService executorService = Executors.newCachedThreadPool();
        final Semaphore semaphore = new Semaphore(threadTotal);
        final CountDownLatch countDownLatch = new CountDownLatch(clientTotal);
        for (int i = 0; i < clientTotal; i++) {
            executorService.execute(() -> {
                try {
                    semaphore.acquire();
                    add();
                    semaphore.release();
                } catch (Exception e) {
                    log.error("exception", e);
                }
                countDownLatch.countDown();
            });
        }
        countDownLatch.await();
        executorService.shutdown();
        log.info("count:{}", count);
    }

    private static void add() { count++; }
```

用Java多线程模拟**5000**次请求，

最多**200**个并发执行的线程数

当于模拟最多**200**个用户的同时发起请求的行为

典型问题 -- 数据不一致

四次执行的结果不一致

```
15:51:41.119 [main] INFO com.mmall.concurrency.example.count.CountExample1 - count:4997
15:52:17.449 [main] INFO com.mmall.concurrency.example.count.CountExample1 - count:4992
15:52:26.056 [main] INFO com.mmall.concurrency.example.count.CountExample1 - count:4988
15:52:34.341 [main] INFO com.mmall.concurrency.example.count.CountExample1 - count:4990
```

采用以下的方案虽然是安全的，但是**不适用于高并发**时的场景

因为这样相当于用户的每次请求都要等待上一个请求完成

```
public static int threadLocal = 100;
```

```
public static AtomicInteger count = new AtomicInteger( initialValue: 0);
```

```
private static void add() {
    count.incrementAndGet();
    // count.getAndIncrement();
}
```

方案一：AtomicInteger

```
private synchronized static void add() { count++; }
```

方案二：加锁

典型问题 -- 服务雪崩

查询一个一定不存在的数据，由于缓存是不命中时需要从数据库查询，查不到数据则不写入缓存，这将导致这个不存在的数据每次请求都要到数据库去查询，造成缓存穿透。

解决一：缓存空对象

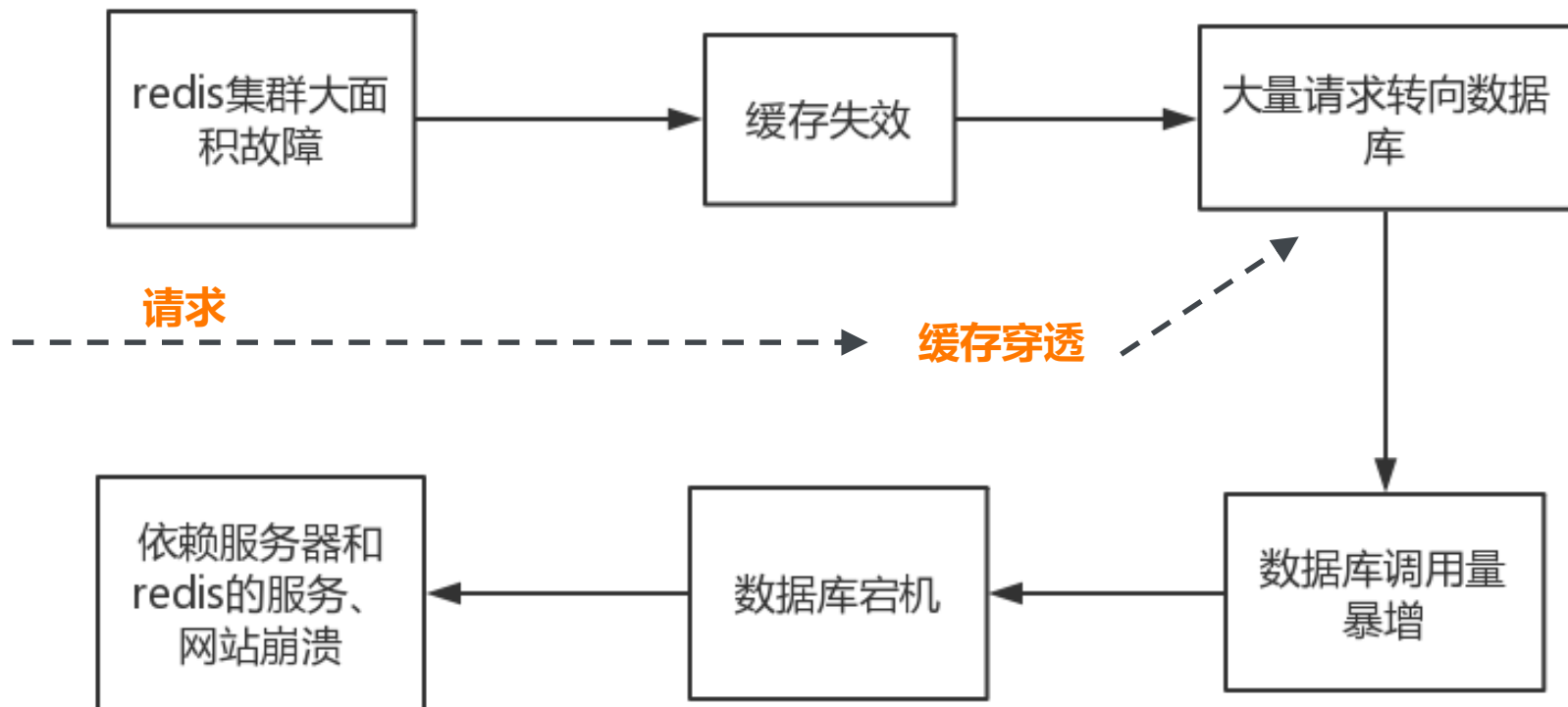
```
cacheValue = GetProductListFromDB();  
if (cacheValue == null) {  
    cacheValue = string.Empty;  
}  
CacheHelper.Add(cacheKey, cacheValue,  
cacheTime);  
  
return cacheValue;
```

解决二：过滤器

对所有可能查询的参数以hash形式存储，在控制层先进行校验，不符合则丢弃。

布隆过滤器 (Bloom Filter)

典型问题 -- 服务雪崩



2

高并发为什么难以解决？

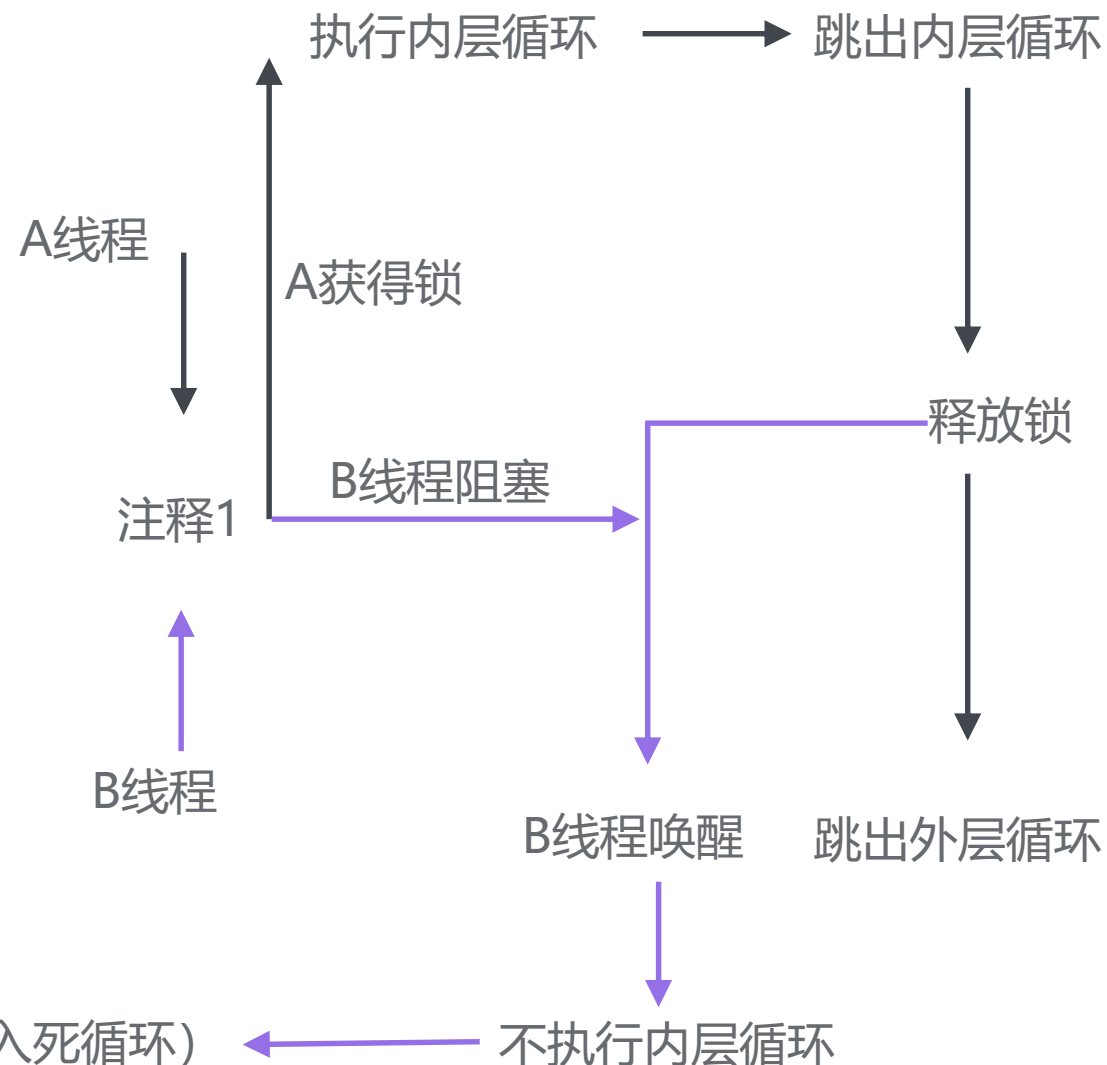
多线程难以调试

乍一看，没问题，**黑线**是A线程，**蓝线**是B线程

```
private final AtomicLong getAtomicLong(String key) {  
    AtomicLong atomicLong = tempData.get(key);           // 注释1  
    while (null== atomicLong) {                           // 注释2  
        try {  
            lock.lock();                                  // 注释3  
            while (null== tempData.get(key))              // 注释4  
            {  
                atomicLong = new AtomicLong(0);  
                tempData.put(key, atomicLong);  
            }  
        } finally {  
            lock.unlock();  
        }  
    }  
    return atomicLong;  
}
```

if (null == (atomicLong = tempData.get(key))) **正确**

无法跳出外层循环（陷入死循环）



大流量难以重现

测试 vs 现实

淘宝是不算是一个耦合性很弱的一个系统，它很难去做到说对整个系统去做一次大型的压力测试，因为这个测试只能当你有现实的访问请求你才知道的。设想一下，你要把整个淘宝复制一份，然后又要去模拟那么多用户真实的访问请求，这是很难做到的。

独立测试，分析整体

但是我们自己有一些经验，比如说我们可以把每个系统独立开来，对他进行做性能测试，然后我们可以对整个核心系统去做分析。

压力测试发现短板

做分析什么？找到短板，然后对它的短板进行压测，然后通过这一系列的压测的结果来分析你整个系统的支撑量是多少，这其实就是由那个短板决定的。那你那个短板是什么？你一定要去发现。而对这种单个系统，你如何去模拟线上的压力，也是一个很重要的研究课题，比如说很简单一个道理，你用户中心要做压测，你怎么把它真实的现状压测出来？那我们的性能测试团队，就是经过了半年摸索，建立了这样的一套大型的系统压测模型。

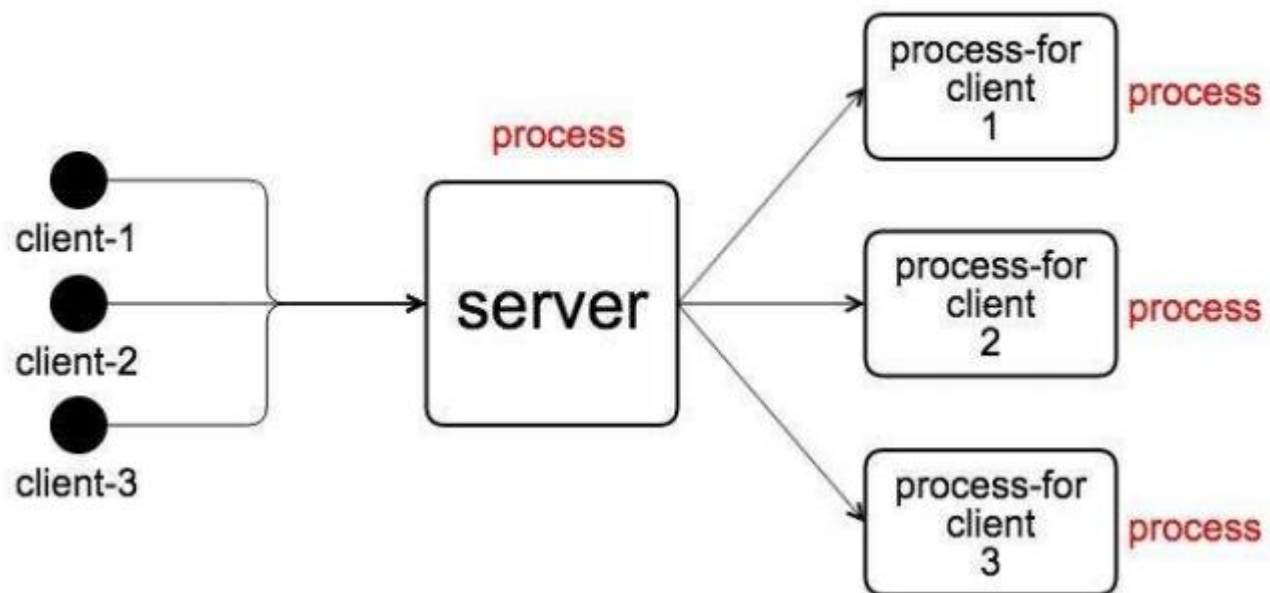
C10K问题

◆ C10K 问题所引发的“想当然”

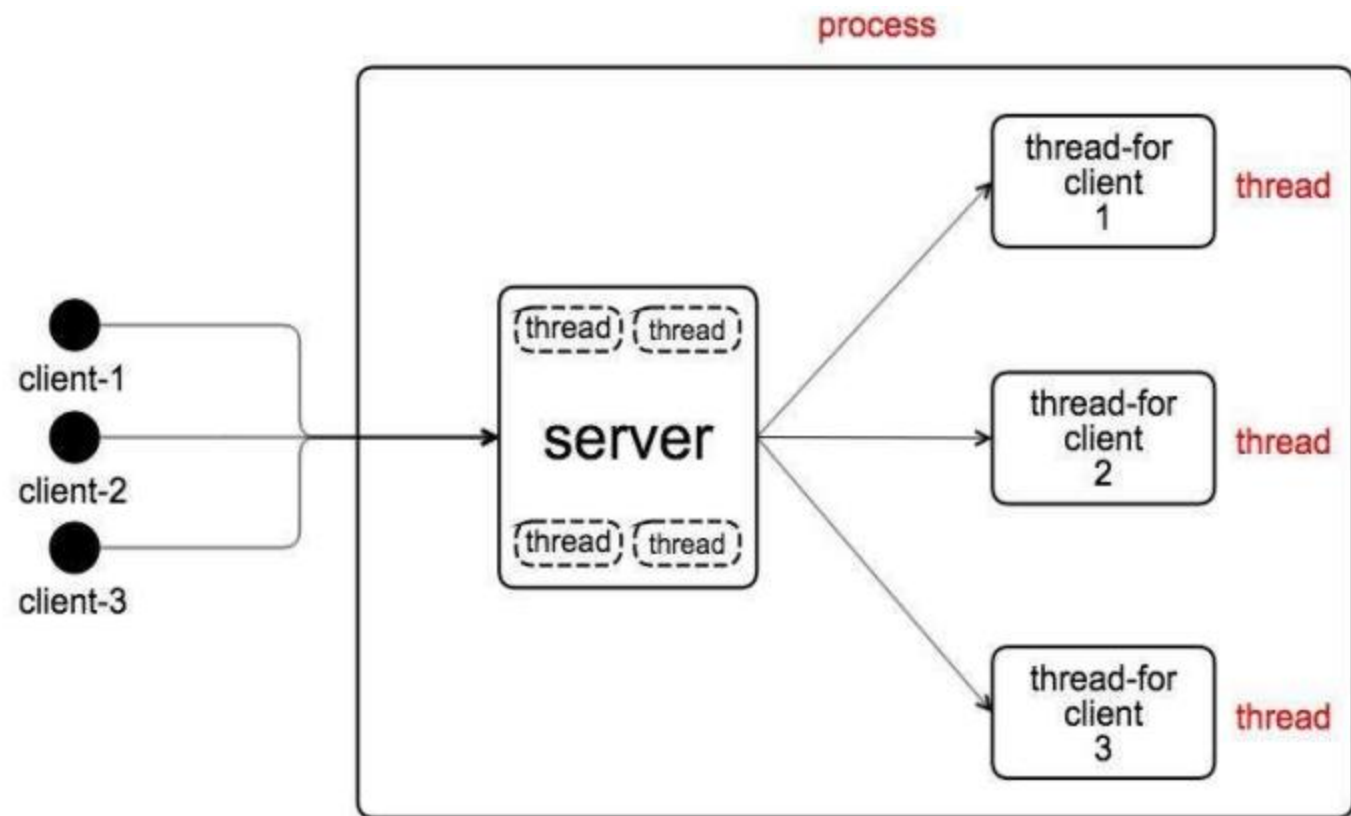
在安全领域有一个“最弱连接”^①（Weakest link）的说法。如果往两端用力拉一条由很多环（连接）组成的锁链，其中最脆弱的一个连接会先断掉。因此，锁链整体的强度取决于其中最脆弱的一环。安全问题也是一样，整体的强度取决于其中最脆弱的部分。

C10K 问题的情况也很相似。由于一台服务器同时应付超过一万个并发连接的情况，以前几乎从未设想过，因此实际运作起来就会遇到很多“想当然编程”所引发的结果。在构成服务的要素中，哪怕只有一个要素没有考虑到超过一万个客户端的情况，这个要素就会成为“最弱连接”，从而导致问题的发生。

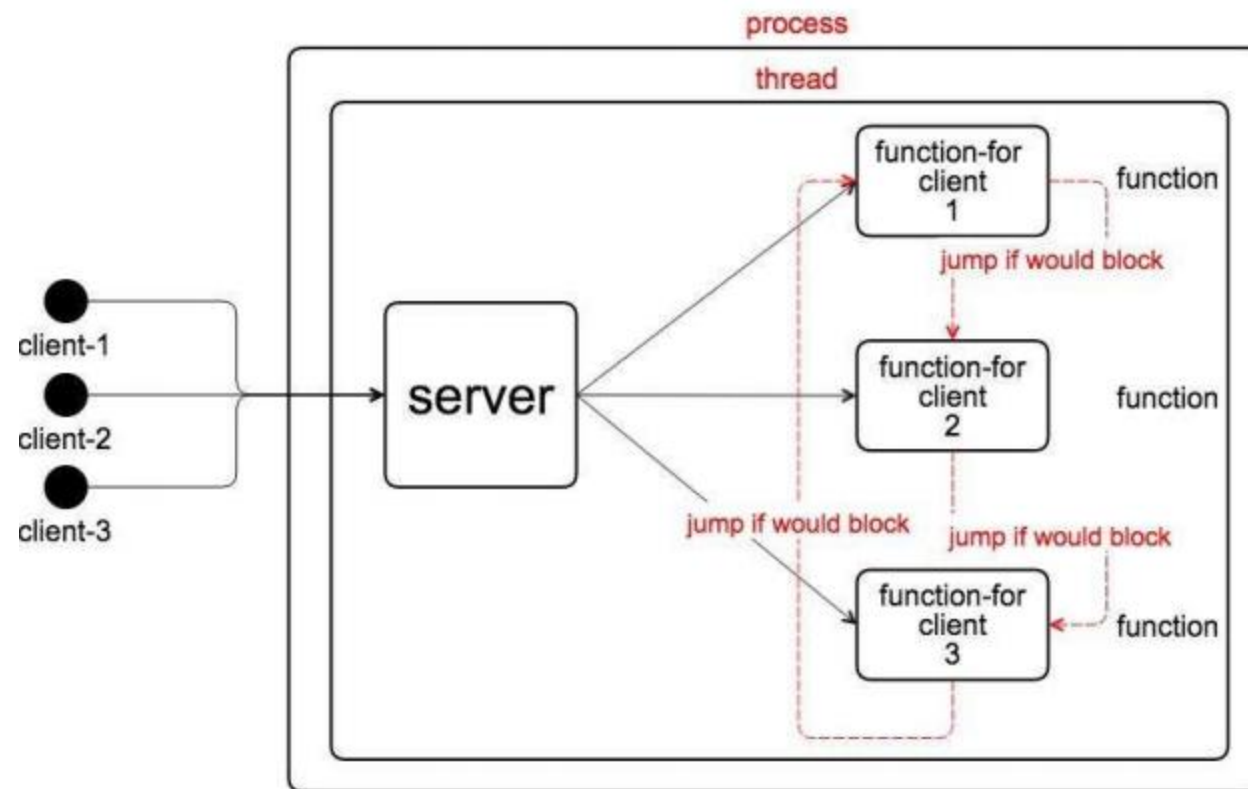
C10K问题 -- 多进程模型



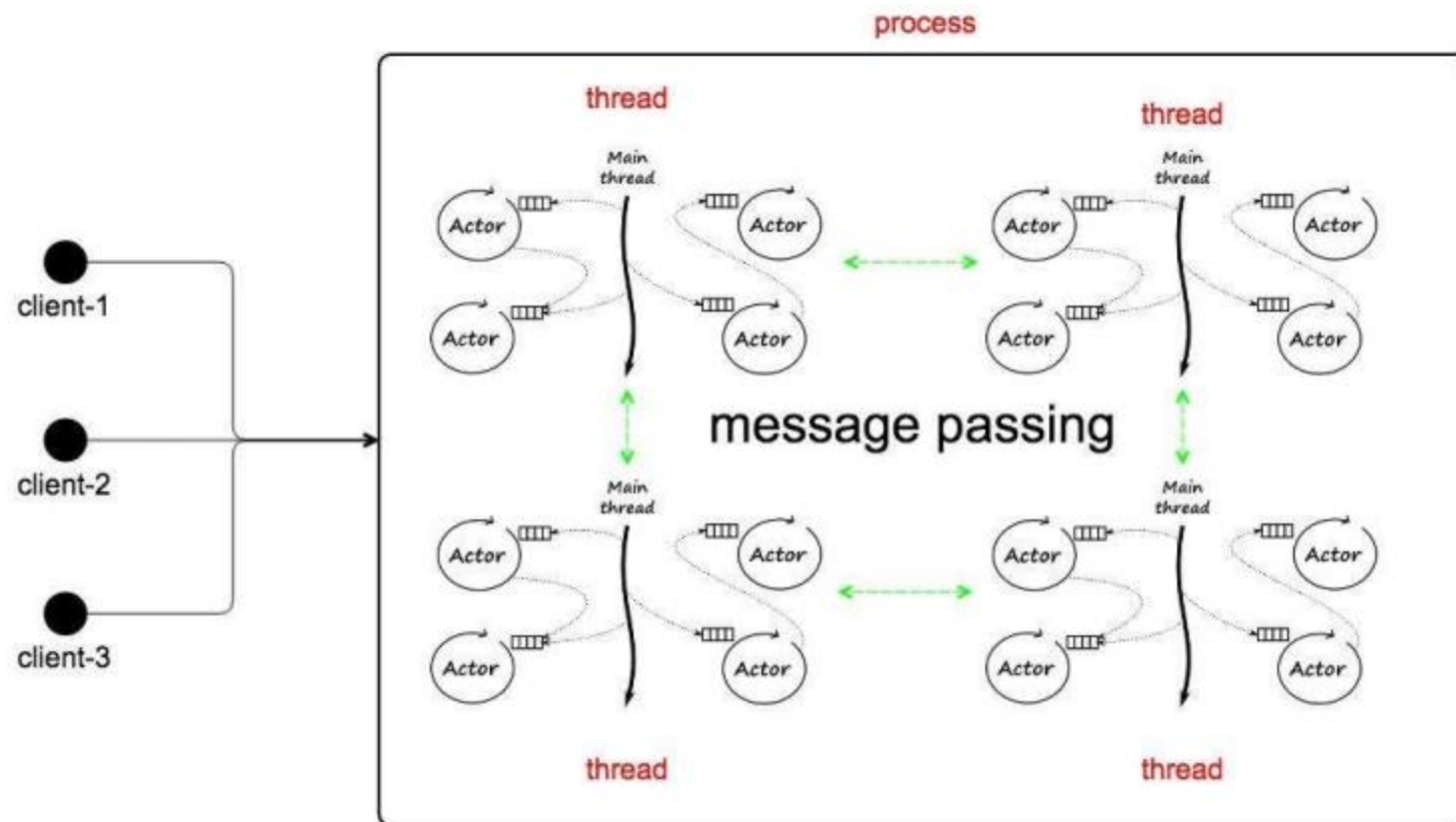
C10K问题 -- 多线程模型



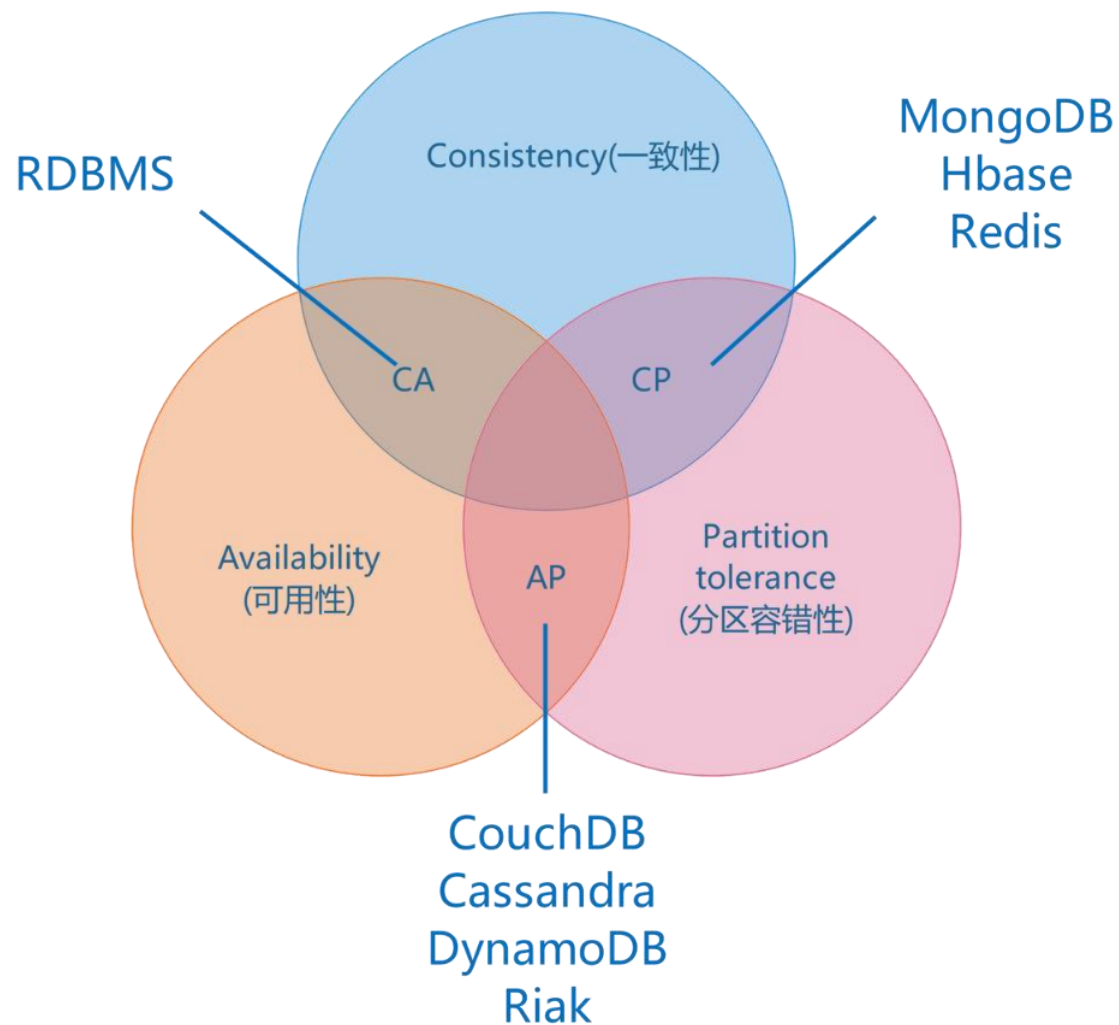
C10K问题 -- 事件驱动模型



C10K问题 -- Actor模型



分布式系统的CAP问题



在一个分布式系统中，一致性、可用性、分区容错性，三者不可兼得

SQL 支持**ACDI**的强事务机制
NoSQL 注重性能与拓展性，非事务性

- Atomicity 原子性
- Consistency 一致性
- Durability 持久性
- Isolation 隔离性

3

天猫双十一背后发生了什么？

2017天猫双11数据：交易峰值**32.5万/秒**，支付峰值**25.6万/秒**，
数据库处理峰值**4200万次 / 秒**

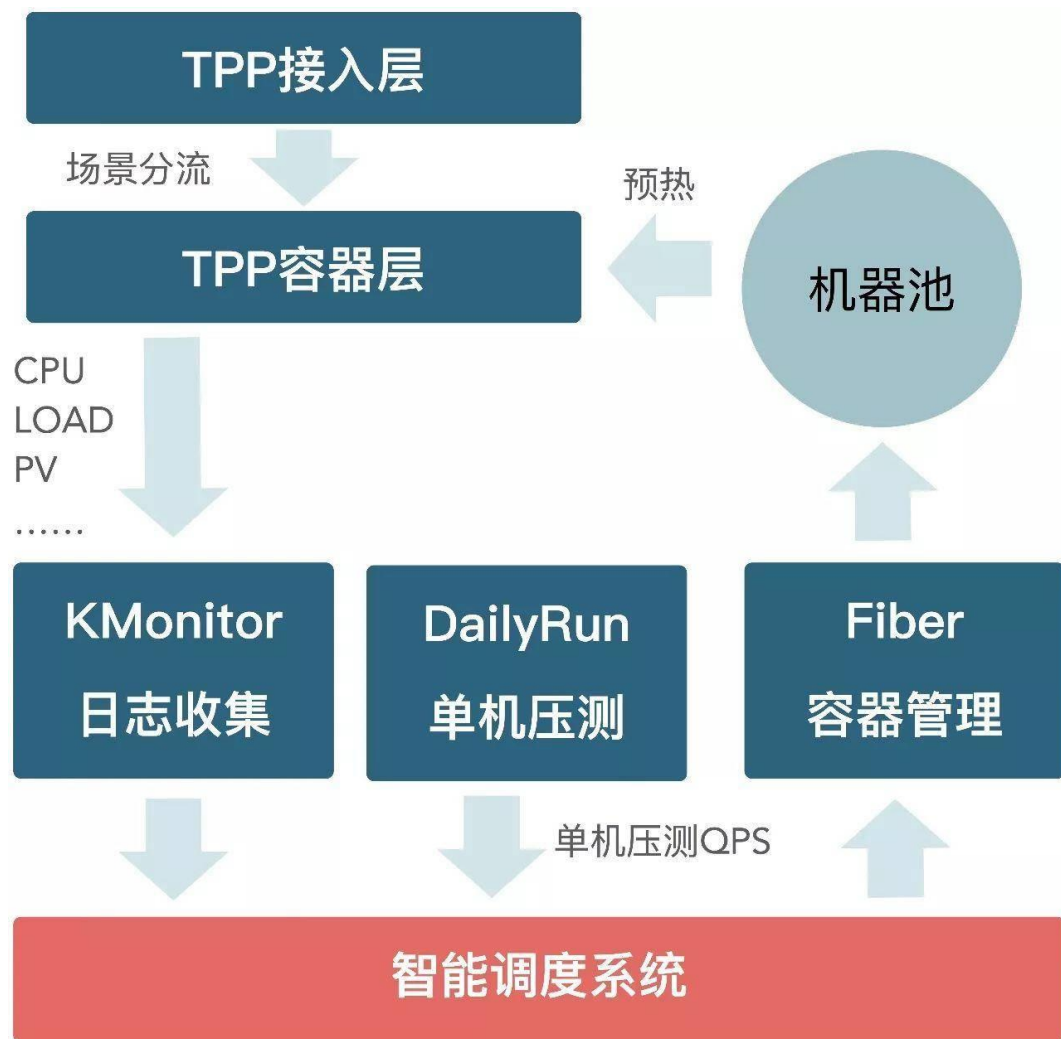
2015年峰值交易量**14万次/秒**

[一张图展示双十一背后的技术](#)

资源自动调度系统

TPP(Taobao Personalization Platform, 也称阿里推荐平台)

TPP智能调度系统以每个集群（一组机器，承载一个或多个场景）的CPU利用率，LOAD，降级QPS，当前场景QPS，压测所得的单机QPS为输入，综合判断每个集群是否需要增加或者减少机器，并计算每个场景需要增减机器的确切数目，输出到执行模块自动增减容器。



资源自动调度系统

利用每个集群当前的运行状态计算每个场景需要的机器数目 N_i , 需要加机器为正值, 需要缩容为负值。

$$N_i = F(CPU, LOAD, \text{异常}QPS, \text{真实}QPS, \text{分级压测}QPS, C_i) \quad \text{其中} C_i \text{为该场景当前机器数} \quad (1)$$

确定每个机器的扩缩容的数目, 寻找最优解, 优化目标为, 即使得CPU尽可能的均衡并接近集群目标负载, 如OptimalCPU=40%为集群最佳状态:

$$\min \sum w_i * (CPU_i - OptimalCPU) \quad (2)$$

$$\text{其中} \quad CPU_i = \frac{\text{当前}QPS}{\text{压测}QPS * (C_i + N_i)} \quad (3)$$

资源自动调度系统

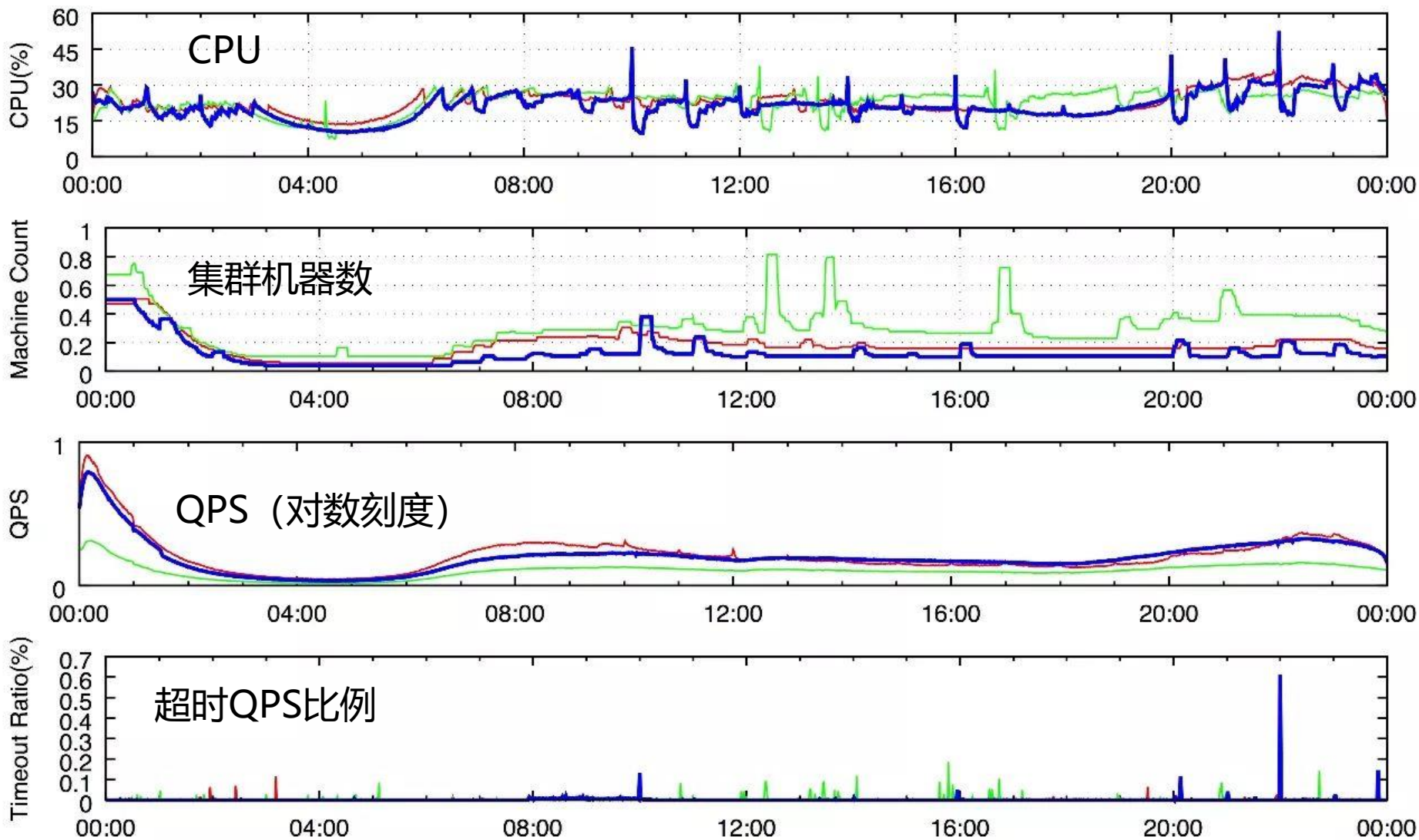
双十一当天的数据

取自四个场景

错峰资源调度

自动扩容保证脉冲

流量



消息中间件RocketMQ

第一代，推模式，数据存储采用关系型数据库。

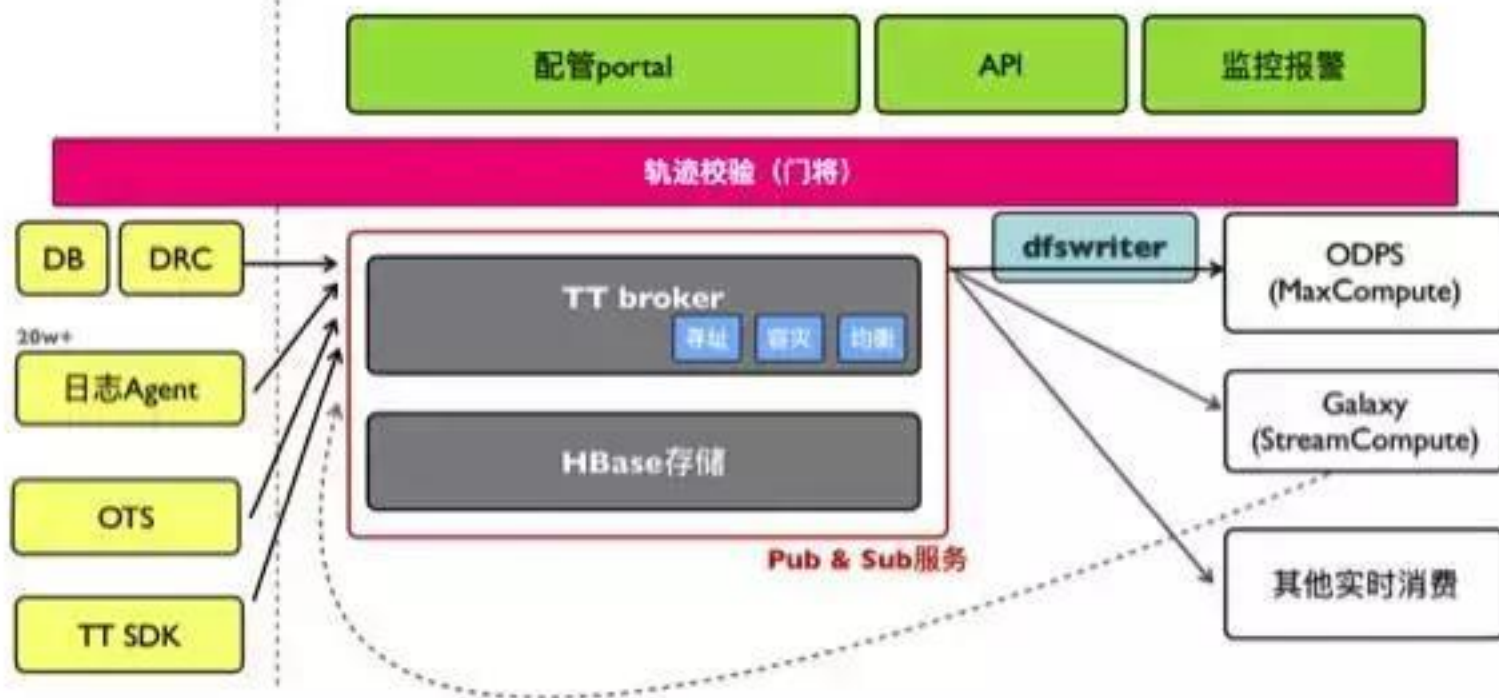
第二代，拉模式，自研的专有消息存储。能够媲美Kafka的吞吐性能，但考虑到淘宝的应用场景，尤其是其交易链路等高可靠场景，消息引擎并没有一位的追求吞吐，而是将稳定可靠放在首位。

2011年研发了以拉模式为主，兼有推模式的高性能、低延迟消息引擎RocketMQ，并在2012年进行了开源。



大数据/云计算平台

TimeTunnel (TT) 在阿里巴巴集团内部是一个有着超过6年历史的实时数据总线服务，它是前台在线业务和后端异步数据处理之间的桥梁。从宏观方面来看，开源界非常著名的Kafka+Flume的组合在一定程度上能够提供和TT类似的基础功能。

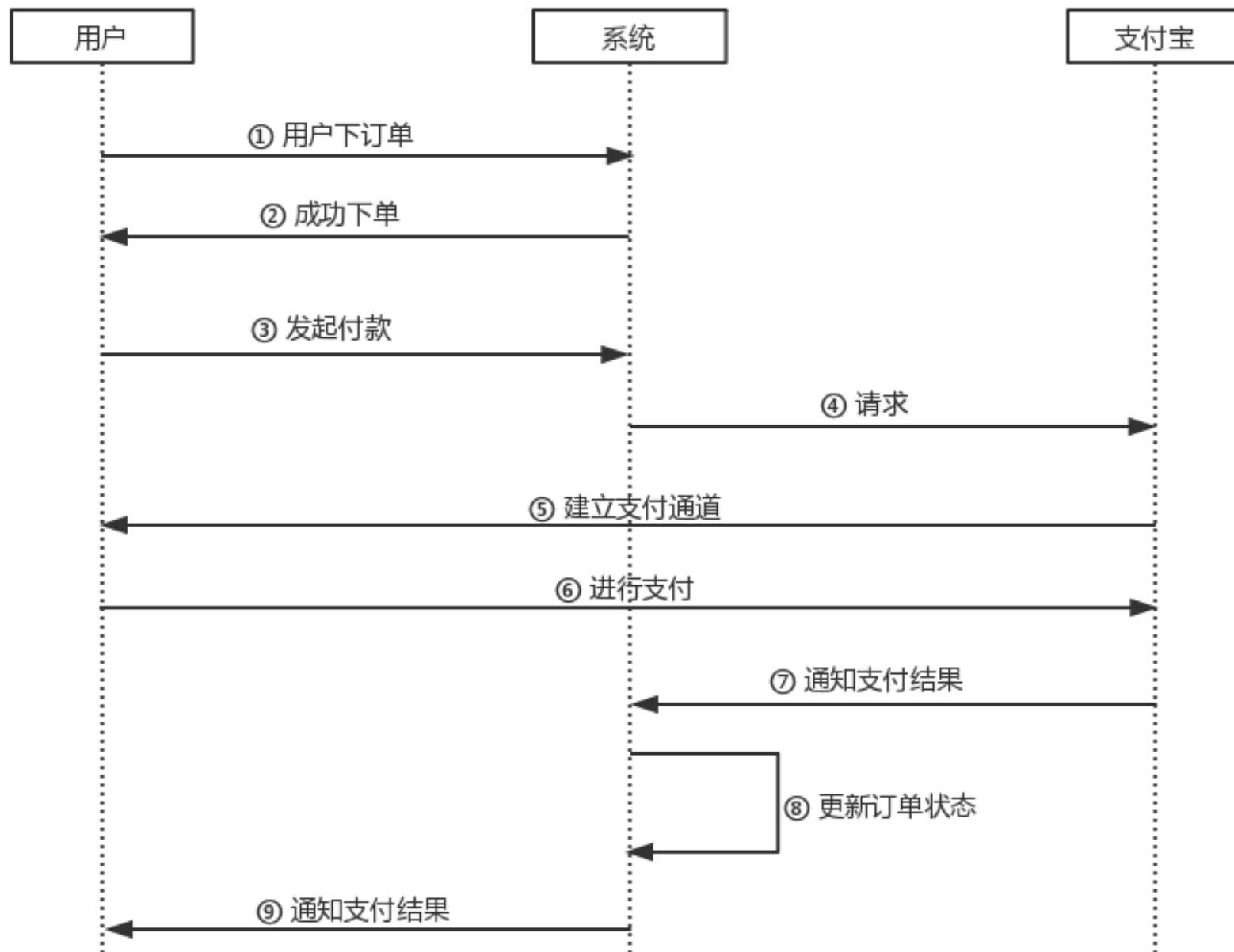


4

那些年，我们经历过的高并发

并发造成的订单号重复

在2018年4月份的时候控制台下单出现订单号重复，后来采用redis分布式锁解决了这个问题



消息服务的限流

- 按特定的速率向令牌桶投放令牌；
- 将请求的报文进行入队列；
- 取出报文后，当桶中有足够的令牌则报文可以被继续发送下去，同时令牌桶中的令牌量按报文的长度做相应的减少；
- 当令牌桶中的令牌不足时，报文将不能被发送，只有等到桶中生成了新的令牌，报文才可以发送。这就可以限制报文的流量只能是小于等于令牌生成的速度，达到限制流量的目的。

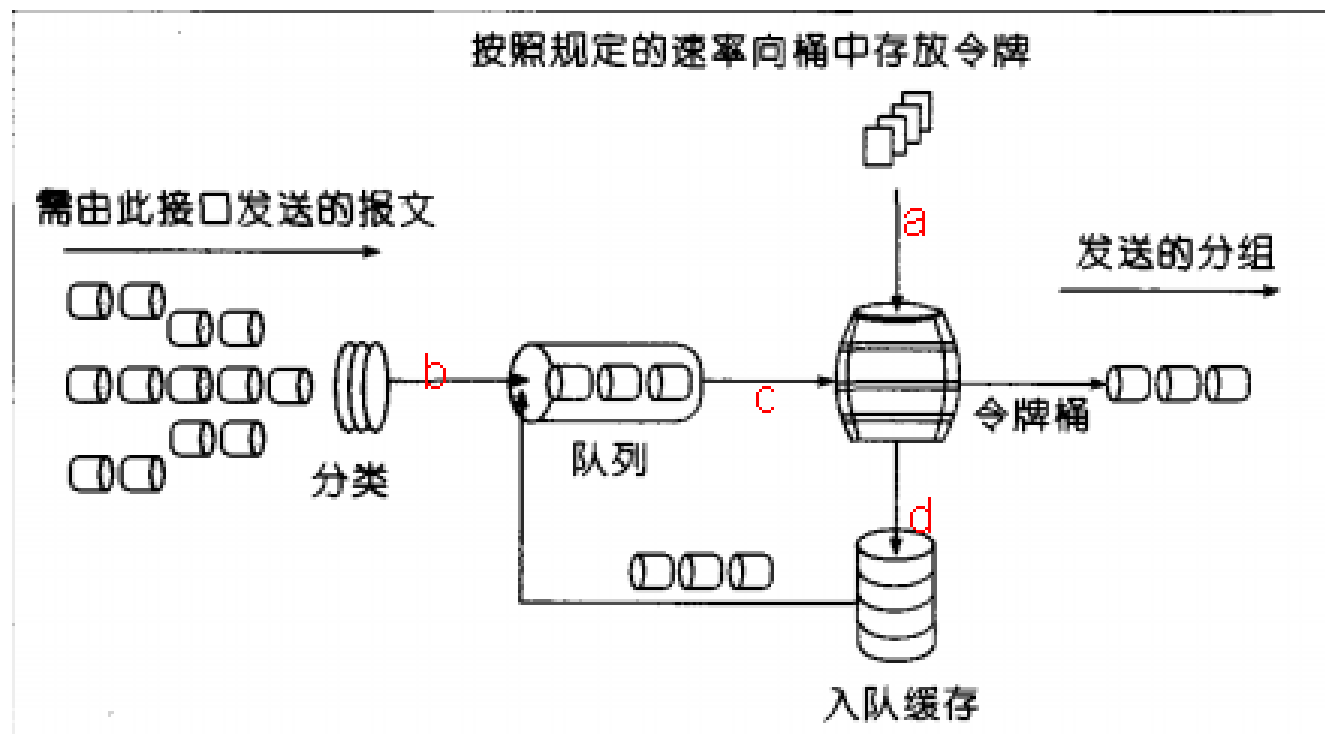


图 3: LR 进行流量控制示意图

5

有没有一套完整的解决方案？

工具

第三方工具

在JDK的开发包中，除了大家熟知的java.exe、javac.exe、javadoc.exe外，还有一系列辅助工具。这些工具在**JDK安装目录下的bin**目录中。

合适的测试用例应该具有：
代表性、覆盖性、可再现性

ab

loadRunner

jmeter

jstack

jstat

jmap

jconsole

合适的测试用例

工具 -- jmeter

Jmeter添加测试计划:

- ① TestPlan→线程（用户）→线程组
- ② 添加取样器→Http请求→设置参数

```
@Controller
@Slf4j
public class TestController {

    @RequestMapping("/test")
    @ResponseBody
    public String test() { return "test"; }
}
```

HTTP请求

名称: HTTP请求

注释:

基本 高级

Web服务器

协议: http 服务器名称或IP: localhost 端口号: 8080

HTTP请求

方法: GET 路径: /test 内容编码:

☐ 自动重定向 ☒ 跟随重定向 ☒ 使用 KeepAlive ☐ 对POST使用multipart / form-data ☐ 与浏览器兼容的头

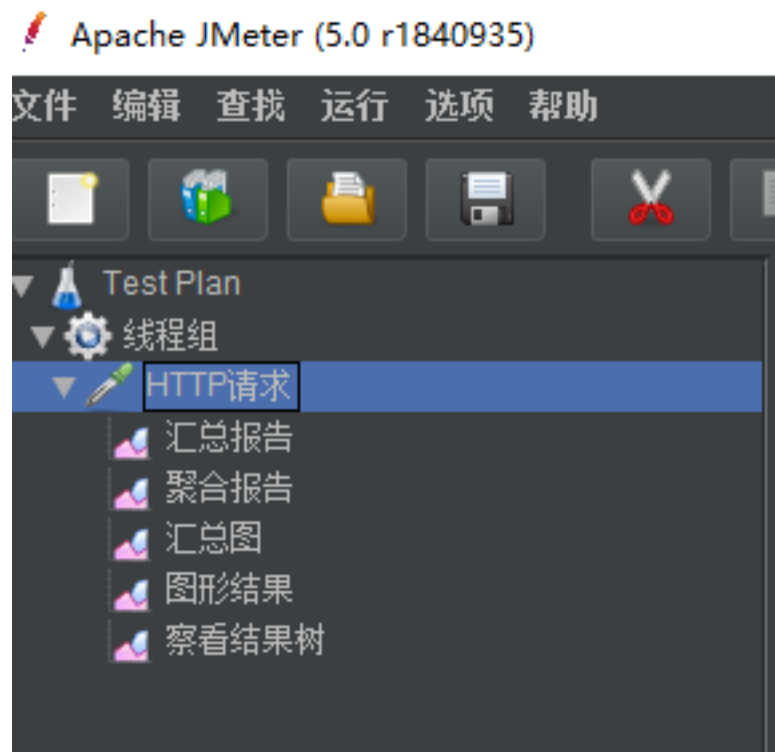
工具 -- jmeter

可添加各种监听器

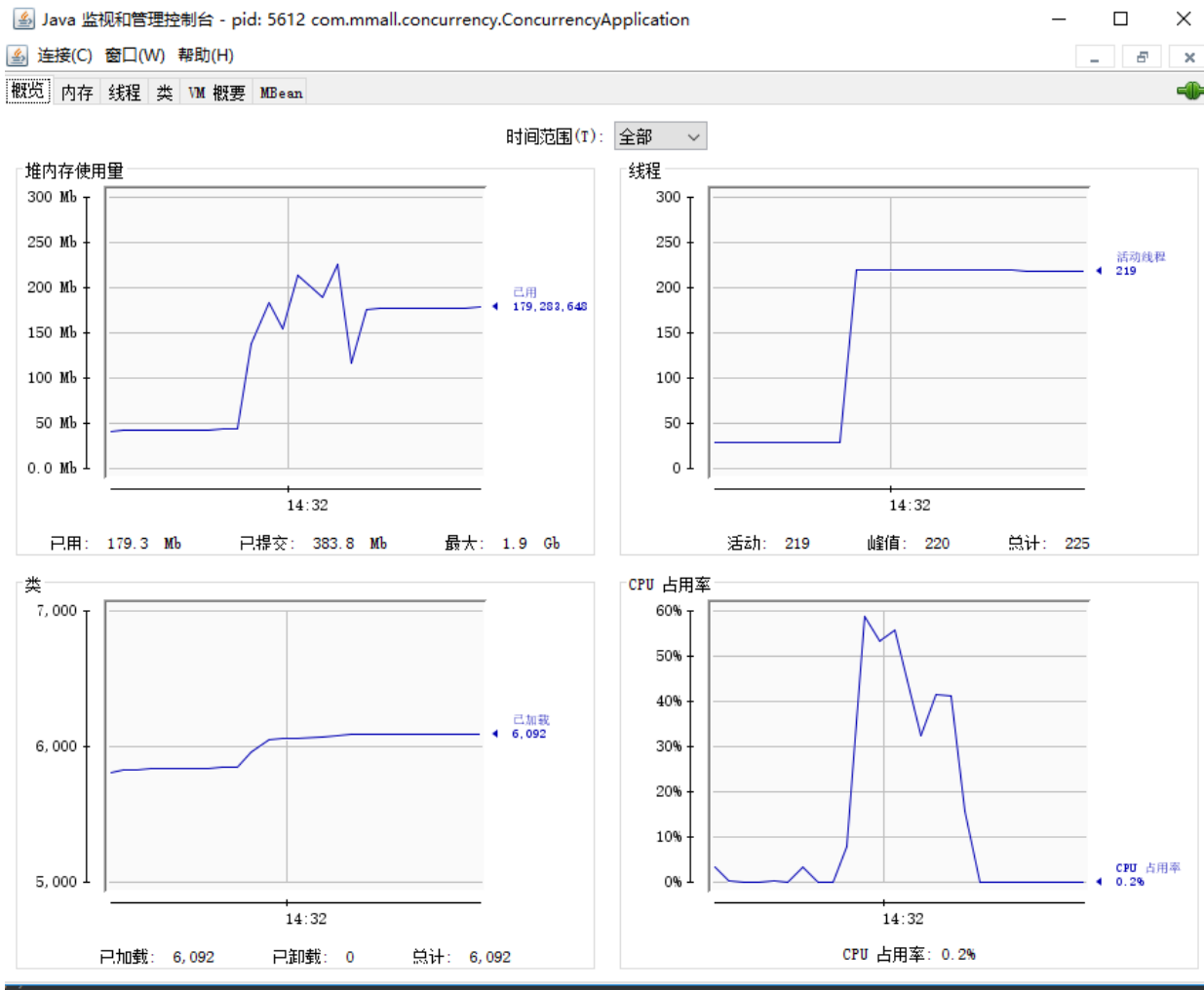
设置线程数为1000（相当于1000个用户）

循环10次访问

所有线程将在1秒（ramp-up）内启动完毕



工具 -- jconsole



- 概况
- 内存监控
- 线程监控
- 类加载情况
- 虚拟机情况
- MBean管理

思路

任务调度分布式elastic-job、
主备curator、
监控报警机制

其他手段

扩容

水平扩容
垂直扩容

NoSQL(Redis/HBase)、
Memcache、GuavaCache

缓存

分库、分表、
多数据源

分库分表

MQ

Kafka、RabbitMQ、
ActiveMQ、
RocketMQ

服务降级的多
种选择、
Hystrix

熔断降级

限流

Guava RateLimiter

应用拆分

服务化Dubbo与微
服务

Spring Cloud

限流算法 (漏桶算法、令牌桶算法)

思路（以秒杀场景举例）

秒杀系统存在的高并发点



寻求优化点

用户请求详情页

CDN

系统时间

无需优化

请求秒杀接口

服务端缓存，如Redis

执行秒杀操作

用**NoSQL（例如Redis）**作为一个原子计数器，记录商品的库存，当用户秒杀该商品时，该计数器便减1；然后记录哪个用户秒杀了该商品，作为一个消息，存储到**分布式MQ**中（例如Alibaba的RocketMQ）；最后由服务端的服务去执行数据库的update操作。

思路（以秒杀场景举例）



执行秒杀操作

使用**RateLimiter**来实现限流，RateLimiter是guava提供的基于令牌桶算法的限流实现类，通过调整生成token的速率来限制用户频繁访问秒杀页面，从而达到防止超大流量冲垮系统。

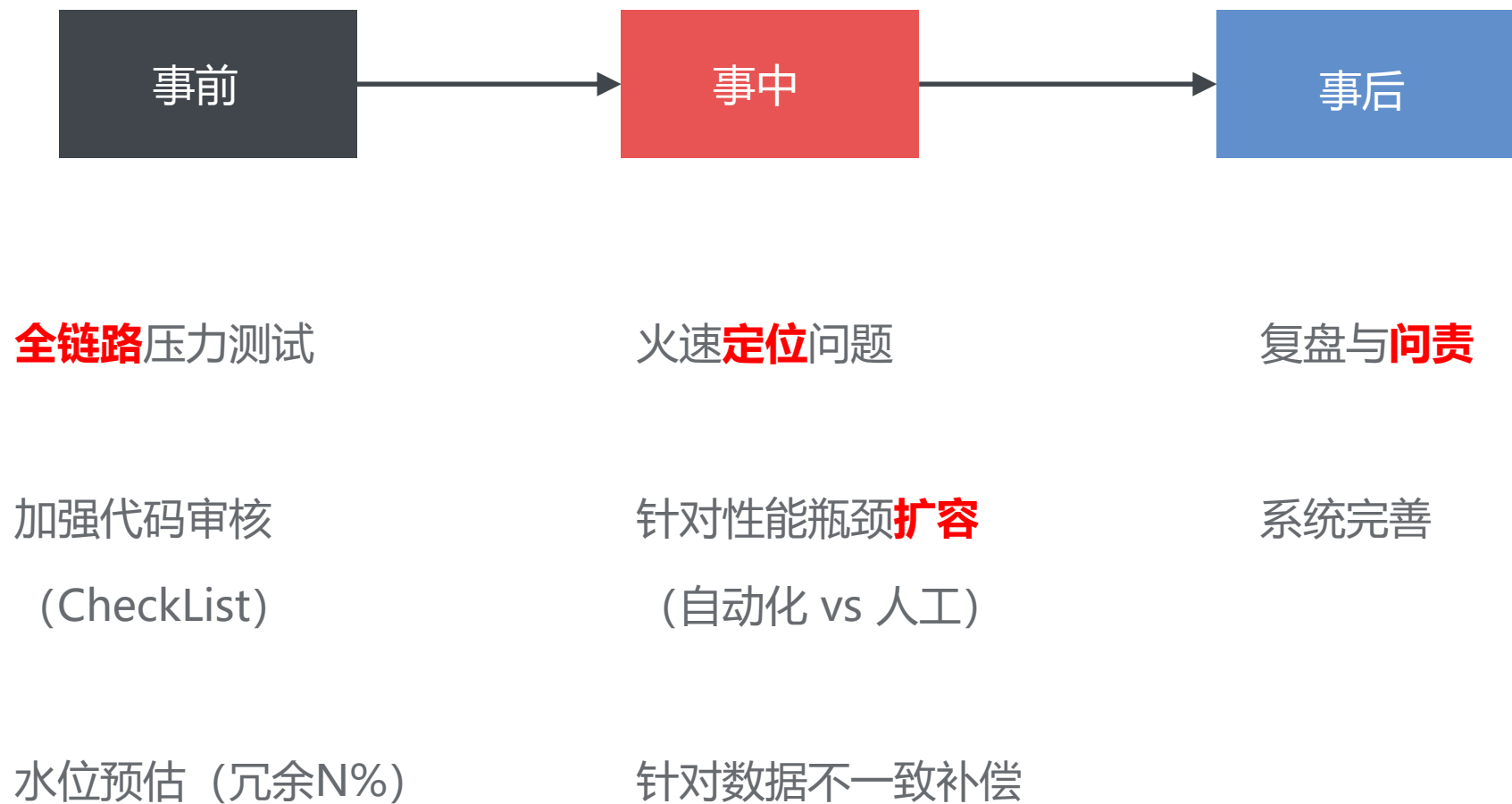
依赖服务

业务拆分，构建微服务体系。比如，说秒杀需要进行手机号验证码的认证，则调用发送短信的微服务

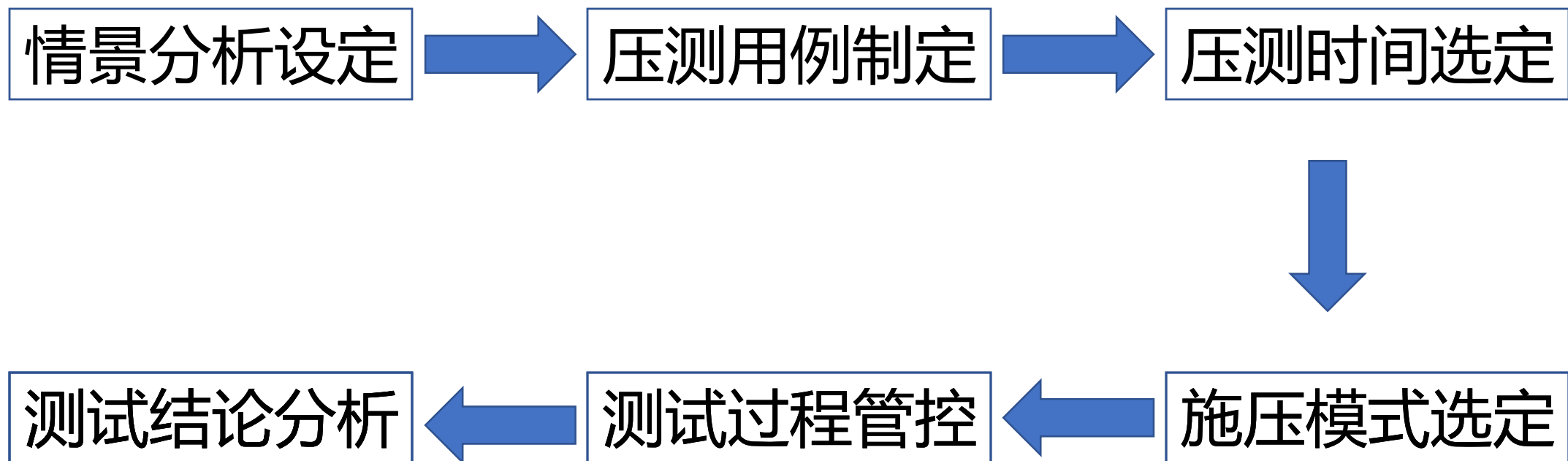
计算资源

提前扩容。秒杀活动开始前，提前进行扩容，根据预设水位分配资源。

维度



压力测试怎么做？



情景分析

分享一个真实的测试案例：

有个做旅游和摄影服务平台的客户要举办一次活动，为本次活动制作了专门的活动页面，在活动页面用户可以报名。那么在短时间内系统到底能撑得住多大的用户并发？

这是**活动运营**和**技术部门**必须提前考虑的问题，因为在去年举办类似活动时就出现了用户大量涌入导致服务不可用的状况，所以首先要帮助用户整理容量测试和规划的工作思路。

场景确定与压测脚本准备：

1. 因为涉及到手机验证场景，在不提供对应API的情况下，建议用户使用万能的验证码或者暂时取消验证码；
2. 是否允许多个手机号同时注册，如果允许我们可用使用固定参数传递，如果不允许，我们可准备对应手机号的测试数据来应对；
3. 短时间内发起大量并发，用户本身是否有安全挡板，如果有，需要把施压节点的IP加入白名单；

模式选定

既然是容量探测，所以整体的施压过程是一个梯度渐进的过程，一般不会上来就是一条直线。这是一直和用户强调的问题，压测的目的绝对不是把系统压垮，压测的目的是通过不断增加的并发来客观评估系统在不同的压力条件下的性能状况；基于此我们定制了施压的**梯度压力模式**

虚拟用户数（VU）	持续时长（分钟）	曲线模式
200	5	坡度
200	2	平行
400	5	坡度
400	3	平行
500	5	坡度
500	5	平行

时间设定

根据系统访问情况，一般会建议用户在凌晨进行压测，此时能够保证对用户的影响最小，也能保证正常用户访问对压测结果的干扰最小。但这个压测时间设定不是绝对的，需要与具体用户的业务场景结合判断。

过程把控

在基本的参数确定之后，就可用根据预定的时间来执行压测任务了，云压测能够进行秒级的数据采集和实时统计分析，能够随时调整压力。随着压力的逐步上升，能够动态呈现系统的性能数据。在逐步加压的过程中，如果性能急剧下降或大量出错，就没有必要继续执行压测任务，此时可以终止任务，也可以下调压力，确保对整个压测过程的把控。

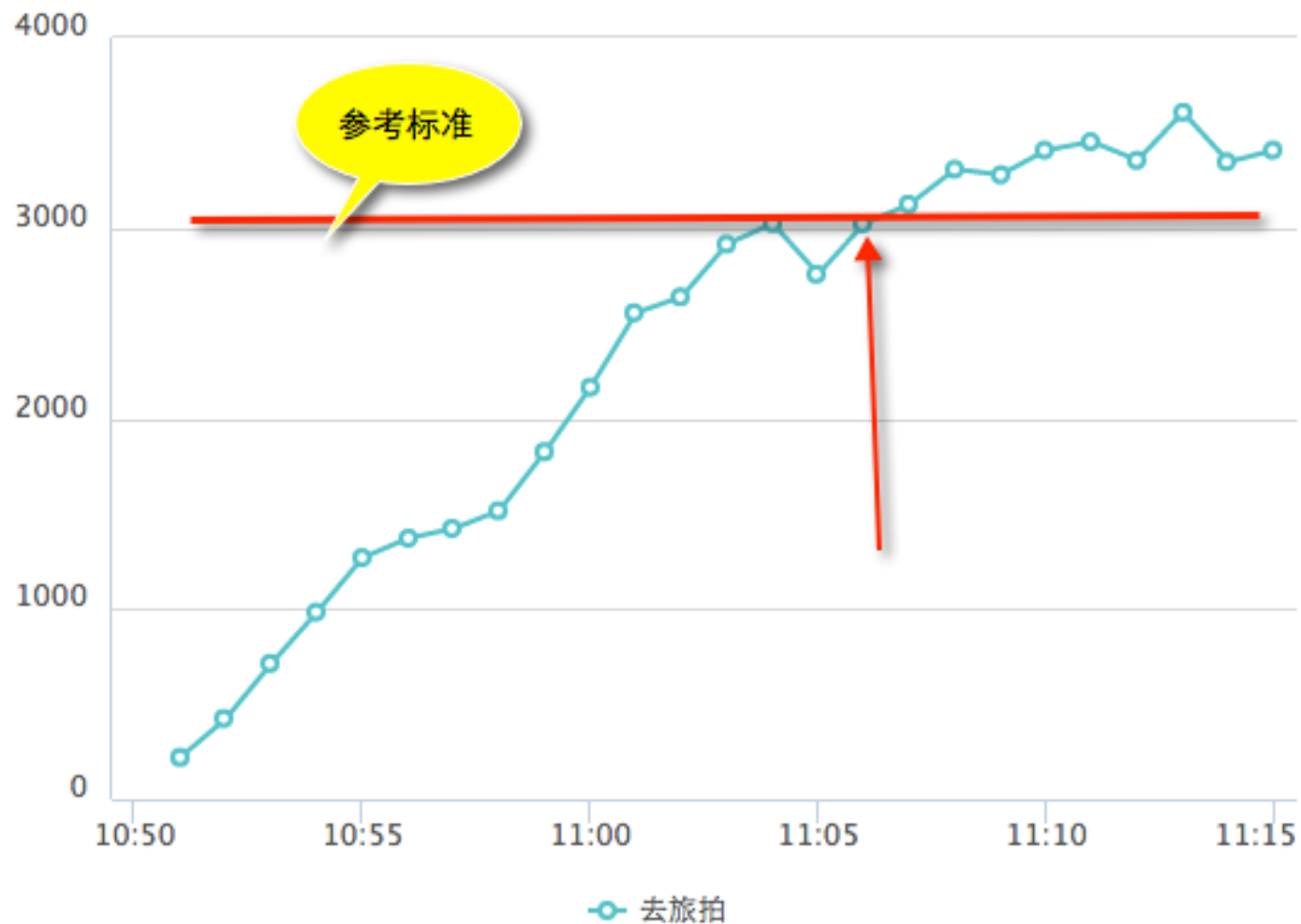
结论分析

被压测的注册接口在360并发用户以下时表现相对良好，在并发用户达到360至500时性能欠佳，说的更直接一点就是

该系统能够支撑360的并发

序号	名称	数值	说明
1	每秒钟事务数（TPS）	132	系统每秒钟处理的事务数。
2	最大并发用户数	500	——
3	平均响应时间	2383ms	——
4	总事务数	197716	压测时段内处理的所有事务的总和
5	失败事务数	4644	在压力测试过程中，发生 400、404、500、502 等 http 错误的事务。 本次集中在 600、601、604 错误；
6	失败率	2.34%	一段时间内事务失败数占事务总次数百分比
7	错误事务数	0	一段时间内事务错误数占事务总次数百分比
8	错误率	0%	在压力测试过程中，断言验证失败的事务。

事务响应时间变化 (ms)



事务响应时间趋势：随着应用访问用户量增加，在并发用户达到420的时候，系统事务处理的时间也显著上升（大于参考标准3000ms），且响应时间在以后一直没有下降。

事务处理性能趋势：当压力稳步上升后，应用事务处理能力保持平稳，基本维持在每秒钟处理130个左右事务，该数值基本能保障并发用户注册的需求。

怎么预估系统水位？

一、经典公式1：

一般来说，利用以下经验公式进行估算系统的平均并发用户数和峰值数据

1) 平均并发用户数为 $C = nL/T$

2) 并发用户数峰值 $C' = C + 3\sqrt{C}$

C 是平均并发用户数， n 是login session的数量， L 是login session的平均长度， T 是考察的时间长度

C' 是并发用户数峰值

举例1，假设系统A，该系统有3000个用户，平均每天大概有400个用户要访问该系统（可以从系统日志中获得），对于一个典型用户来说，一天之内用户从登陆到退出的平均时间为4小时，而在一天之内，用户只有在8小时之内会使用该系统。那么，

平均并发用户数为： $C = 400 \times 4 / 8 = 200$

并发用户数峰值为： $C' = 200 + 3\sqrt{200} = 243$

二、通用公式2:

对绝大多数场景，我们用（用户总量/统计时间）*影响因子（一般为3）来进行估算并发量。

比如，以乘坐地铁为例子，每天乘坐人数为5万人次，每天早高峰是7到9点，晚高峰是6到7点，根据8/2原则，80%的乘客会在高峰期间乘坐地铁，则每秒到达地铁检票口的人数为 $50000 * 80\% / (3 * 60 * 60) = 3.7$ ，约4人/S，考虑到安检，入口关闭等因素，实际堆积在检票口的人数肯定比这个要大，假定每个人需要3秒才能进站，那实际并发应为 $4 \text{人/s} * 3 \text{s} = 12$ ，当然影响因子可以根据实际情况增大！

三、根据PV计算公式：

比如一个网站，每天的PV大概1000w，根据2/8原则，我们可以认为这1000w pv的80%是在一天的9个小时内完成的（人的精力有限），那么TPS为：

$1000w * 80\% / (9 * 3600) = 246.92 \text{ 个/s}$ ，取经验因子3，则并发量应为：

$$246.92 * 3 = 740$$

四、根据TPS估计：

公式为 **$C = (\text{Think time} + 1) * \text{TPS}$**

五、根据系统用户数计算：

并发用户数 = 系统最大在线用户数的8%到12%

6

总结

银弹

短板

感谢聆听!



欢迎讨论